

Programação Orientada a Objetos

Tecnologia em Análise e Desenvolvimento de Sistemas

Kinan Principe

25-05-2026

Contents

1. Programação Orientada a Objetos	3
1.1 Abstração	3
1.2 Programação Estruturada	3
1.3 Gap semântico	3
1.4 POO	3
1.5 Resumo em Tópicos	3
2. Abstração	3
2.1 Classes	3
2.2 Atributos	3
2.3 Contexto ou domínio	3
2.4 Engenharia de Domínio	4
2.5 Programação Orientada a Objetos	4
2.6 Representar as Abstrações	4
2.6.1 Conceitualmente	4
2.6.2 Na Programação Orientada a Objetos	4
2.7 Representação UML (tentar dps com latex)	4
2.8 Em java	4
2.9 Exercício	5
3. Instanciação	6
3.1 Atribuição de objetos de mesma classe	6
3.2 Garbage Collector	6
3.3 Métodos	6

1. Programação Orientada a Objetos

1.1 Abstração

- Redução da complexidade de um objeto para focar em sua funcionalidade.

1.2 Programação Estruturada

- Structs, variáveis, constantes, funções, procedimentos, módulos;
- Estruturas de repetição, estruturas de decisão, vetores, matrizes.

1.3 Gap semântico

- Gap: a falta de algo para atingir um determinado objetivo.

1.4 POO

- Funções, procedimentos e módulos tornam-se *métodos*.
- Todo o restante tornam-se *atributos* dos objetos.

1.5 Resumo em Tópicos

- Abstração (objetos e relações)
- Programação Estruturada
- Engenharia reversa?? n entendi oq ele quis dizer aq
- Conceitos do paradigma OOP
 - Partes de PE são renomeadas para *métodos* e *atributos*.

2. Abstração

- Algo tangível ou intangível que pode ser representado em um determinado contexto.
- A abstração “livro” é diferente no contexto da biblioteca e da livraria, pois:
 - Biblioteca: livros são emprestados e devolvidos depois;
 - Livraria: livros são comprados.

2.1 Classes

- Categorias de objetos.

2.2 Atributos

- Aquilo que vai determinar as características de uma classe de objetos.

2.3 Contexto ou domínio

- Ambiente/espço limitado.

2.4 Engenharia de Domínio

- Técnica de programação que permite desenvolver software para uma área inteira.

2.5 Programação Orientada a Objetos

- Representa-se as abstrações em um contexto por meio de objetos.

2.6 Representar as Abstrações

2.6.1 Conceitualmente

- Na Programação Orientada a Objetos, considera-se as *abstrações* objetos.
- Contexto mundo:
 - Pessoas
- Contexto universidade:
 - Alunos (contexto);
 - Professores (contexto);
 - TAEs (contexto).
- Professor (classe):
 - Prof. Antônio (objeto).
- Alunos (classe):
 - João (objeto);
 - Maria (objeto);
 - Fernando (objeto).

2.6.2 Na Programação Orientada a Objetos

- Representadas por meio de classes e objetos.

2.7 Representação UML (tentar dps com latex)

2.8 Em java

Classes e atributos

```
class Candidato { // nome da classe
    String noInscricao; // atributo
    String nome; // atributo
}

class processoSeletivo {
    public static void main(String[] args) { // método de inicio
        // moléculas, moléculas...
```

```
Candidato amarildo; // referência não instanciada
Candidato karina; // referência não instanciada
    }
}
```

2.9 Exercício

- Apostila capítulos de 1 a 3, tipagem em Java;
- Defina contexto, descrevam abstrações, use um glossário para definir abstrações;
- Fazer 5 classes;
- Traduzir para UML e depois Java.

3. Instanciação

3.1 Atribuição de objetos de mesma classe

- Ao atribuir dois objetos de mesma classe, um se tornará referência do outro e o objeto que deixar de ser referenciado será automaticamente desalocado pelo *garbage collector*.
- Exemplo:

```
class processoSeletivo {
    public static void main(String[] args) {
        Candidato amarildo;
        amarildo = new Candidato();

        Candidato karina = new Candidato();
        Candidato candidato;

        amarildo.nome = "Amarildo";
        amarildo.nInscricao = 113456;

        karina.nome = "Karina";
        karina.nInscricao = 202507;

        candidato = karina;
        candidato.nome = "Ivo";

        System.out.println(karina.nome); // output: Ivo
    }
}
```

3.2 Garbage Collector

- Mecanismo que identifica instanciações sem referência e as elimina (desaloca) para otimização;
- Melhora desempenho em dispositivos com limitação de memória.

3.3 Métodos

- Todo método tem um *efeito* e uma *assinatura*.
- Exemplo:

```
public static void main(String[] args) {
    Candidato karina = new Candidato();

    karina.mostrarNome();
}
```

```
}

public class Candidato {
    public String nome;
    public int nInscricao;

    public void mostrarNome() {
        System.out.println("Nome      : "this.nome);
        System.out.println("Inscricao: "this.nome);
    }
}
```